Nicolás Andrés Cano Leal

Backend Engineer → SRE | Python · Django · Kubernetes · Terraform · CI/CD

Rancagua, Chile (remote) · nicolas.cano.leal@gmail.com · +56 9 9411 3328 · github.com/NicolasAndresCL
· linkedin.com/in/nicolas-andres-cano-leal · nicolasandrescl.pythonanywhere.com

SUMMARY

Backend engineer in Python who builds services with operations in mind, not just code. I am the only developer on an operations team at PedidosYa, where I replace manual processes with tested Python services that run unattended against BigQuery, Google Sheets and internal systems. I work with Django and DRF and take every project to production with CI/CD, containers and infrastructure as code. My goal is Site Reliability Engineering: systems that scale, are observable and recover on their own.

EXPERIENCE

PedidosYa — Live Performance Agent (LiveOps / BizOps)

Mar 2026 – present

Rancagua, Chile (remote) · Sole developer on the operations team

Real-time operations automation bot

- Evolved the internal bot across 19 releases (v4.0 → v6.7), backed by 2,800+ automated tests and 650+ commits following professional conventions.
- Orchestrated ~10 concurrent threaded engines (alerts, reporting, watchdogs) and closed a check-then-act race condition between them with an atomic-reservation cooldown.
- Cut Google Sheets calls from 21 round-trips to 1 with a TTL-cached singleton (~16 min → <1 min) and reduced the global bottleneck from ~37 min to ~10 min.
- Layered design: pure logic isolated from Selenium, network and BigQuery, testable without a browser.

Incident dossier collector

- Built a Django/DRF backend with a NiceGUI UI that gathers each incident's evidence from BigQuery, replacing manual spreadsheet and email work.
- A measured audit found the main data source had dropped to 0.11% coverage; I moved the timeline to a table with 100% coverage and 13 months of retention. 660 tests.

Cancelled-order participant sync

- Service that reads an event stream in BigQuery (read-only) and publishes idempotently to Google Sheets without overwriting analysts' annotations.
- Ports-and-adapters architecture: switching the sink from BigQuery to Sheets was a config change, not a domain change. Decisions recorded as ADRs with UML diagrams.
- Running on 60,000+ rows: diagnosed 923 corrupted cells through statistical proportion and migrated the live sheet's schema with no data loss. 312 tests, 80% coverage.

Python · Django/DRF · NiceGUI · BigQuery · Google Sheets API · Selenium · Pandas · pytest · GitHub Actions

PROJECTS

All in Django — backend with full infrastructure as code

repo

- Django + DRF API for five operational domains, with a NiceGUI client UI that talks HTTP only. 284 tests, 80% minimum coverage enforced in CI.
- OpenAPI contract (drf-spectacular) verified by an 82-request Postman collection that Newman runs in CI against an ephemeral stack; a test fails if any endpoint goes untested.
- 6-job CI (GitHub Actions) and Jenkins CD: v1.0.0 released to GHCR and deployed with a pre-deploy backup and automatic rollback.
- Terraform (EC2 + RDS PostgreSQL) tested without an AWS account using `terraform test` and mock providers; the tests surfaced three latent bugs before the first deploy. Helm for Kubernetes.

Personal portfolio — Django 5.2 LTS + React (in production)

repo

- Migrated production from SQLite to managed PostgreSQL 16 (Neon) with no data loss, after finding that the host's Postgres (v12) was incompatible with Django 5.2.
- Typed config with pydantic-settings, throttling, pagination, `/healthz/`, per-environment HTTPS hardening. Type checking with mypy + django-stubs, 49 tests, ~91% coverage.
- GitHub Actions CI and Jenkins-in-Docker CD (test gate → image → GHCR → deploy).

Reproducible infrastructure with Docker

- n8n-docker-local: n8n with a pinned version, protected external volume, real healthcheck and CI that verifies the service starts.
- ultimate-linux-lab: Ubuntu 24.04 LTS with real systemd in a disposable container, replacing a virtual machine.

Other projects

DRF and GraphQL APIs (JWT, OpenAPI), a Django + React e-commerce, a FastAPI + SQLAlchemy CRUD, a Flask monolith refactored into a REST API, Streamlit dashboards and a scikit-learn regression model. Full catalog on the portfolio site.

TECHNICAL SKILLS

Languages	Python (advanced), SQL, JavaScript, Java (learning), Bash
Backend	Django, Django REST Framework, FastAPI, Flask, SQLAlchemy, pydantic, drf-spectacular (OpenAPI), GraphQL, JWT
Databases	PostgreSQL (Neon, RDS), MySQL, SQLite, BigQuery
DevOps / IaC	Docker (multi-stage, non-root), Docker Compose, GitHub Actions, Jenkins, GHCR, Terraform, Helm, Kubernetes, Linux, AWS (EC2, RDS)
Quality	pytest, coverage, mypy, Postman + Newman, contract testing, ADRs, UML
Automation	Selenium, n8n, Google Sheets API, BeautifulSoup, Pandas, idempotent ETL
Interfaces	React, Vite, NiceGUI, Streamlit, Plotly
Tooling	Git (conventional commits), AI-assisted development (Claude Code + MCP), VS Code

EDUCATION

- **Computer Engineering** — IP Santo Tomás (in progress)
- **Nursing Technician**, Surgical Instrumentation — INACAP, 2014
- 2,500+ hours of self-directed study in Python backend, architecture and DevOps

LANGUAGES

Spanish (native) · English B1+ (fluent technical reading)

CERTIFICATIONS

- **Ultimate Docker** · **Ultimate Java** — Hola Mundo (2026)
- Ultimate Python · Ultimate Git + GitHub · Ultimate n8n — Hola Mundo (2026)
- SQL: Sin Fronteras — Hola Mundo (2025)
- Building LLM Apps (LangChain, Semantic Kernel, Prompt Flow) · GitHub Models — LinkedIn Learning (2026)
- Master Python: PySpark · Advanced Python 2 — LinkedIn Learning (2026)
- Master Python: Django REST · FastAPI · AWS for IT Professionals — LinkedIn Learning (2025)
- Data Analysis with Python — Desafío Latam (2025)